

DiskANN-RSI: Query-Path Optimizations for a Rust-Native Vamana Index, and a Same-Machine Reproducibility Study

Zhanhui Kang
TerseMind

research@tersemind.ai <https://github.com/kangzhanhui>
ORCID: <https://orcid.org/0009-0008-7791-1917>

September 2026

Abstract

We present **DiskANN-RSI**, a Rust-native implementation of the Vamana/DiskANN graph index whose query path was iteratively optimized by an automated research-loop (RSI) framework guided by recent graph-ANN theory (Gollapudi et al., ICML 2025). Under the optimizer’s own evaluation harness, the accumulated query-path changes raise single-thread QPS-at-recall ≥ 0.95 on SIFT1M from 2994.6 to 4641.1 (+55%), with the dominant single change being a cache-resident epoch-stamped visited set using 16-bit stamps. We then re-evaluate the same binary against unmodified baselines (stock Vamana/DiskANN, hnswlib in three configurations, and NSG) under one protocol on one machine. The isolated gains do not transfer uniformly: DiskANN-RSI trails stock Vamana by 9–31% at low search depths and reaches parity at high recall, while building its index $1.9\times$ faster (131.9s vs. 255.0s) and scaling slightly better to two threads ($1.87\times$ vs. $1.75\times$). We analyze why the two evaluation protocols disagree, report several negative results (SQ8 quantization, candidate-queue merge insertion, random-projection reordering, partial-distance early exit), and release code, raw measurements, and runbooks to support reproduction. Our findings quantify a familiar but under-documented hazard: optimizer-in-the-loop harnesses reward changes that do not survive protocol unification, and same-machine cross-system baselines remain necessary.

1 Introduction

Approximate nearest neighbor (ANN) search on graph indexes underlies modern vector retrieval. The Vamana graph, introduced with DiskANN [1], remains a reference design for single-node indexes, and recent work continues to tighten its construction guarantees [2]. Separately, automated research-loop systems (“AI scientists”) [7, 8] have begun to propose, implement, and measure code optimizations with minimal human involvement, raising a practical question: *do gains reported by an optimizer’s own harness survive contact with a unified, same-machine benchmark against independent baselines?*

This paper is both a system description and a reproducibility study. We ported DiskANN’s full component set to Rust (graph index, SSD-resident index with aligned reads, PQ/SQ quantization, label filtering, streaming executor), and let an automated research-and-iteration loop (RSI) optimize the in-memory Vamana query path on SIFT1M. We then froze the resulting binary and benchmarked it against stock Vamana, hnswlib, and NSG on one cloud host under one protocol. Our contributions:

- A Rust-native Vamana/DiskANN implementation with a reproducible, configuration-driven benchmark harness reporting QPS-at-recall operating points (Section 3).
- A set of query-path optimizations — an epoch-stamped visited set with 16-bit stamps, adaptive early termination, exposed tuning levers, and fat link-time optimization (LTO) — with per-change measurements (Section 4).
- A same-machine, single-protocol comparison against stock Vamana, hnswlib (three variants), and NSG, including recall-QPS ladders, thread scaling, build time, and peak memory (Section 5).
- An analysis of where the two evaluations diverge and a documented set of negative results (Sections 5.5–6), with all raw data released.

2 Background

Graph ANN. HNSW [3] builds a hierarchical navigable small-world graph; NSG [4] sparsifies a kNN graph toward the monotonic-relative-neighborhood property; DiskANN’s Vamana [1] uses greedy search plus a robust pruning rule with a slack parameter α , supporting both in-memory and SSD-resident operation. Worst-case analyses of popular implementations [5] and of the DiskANN family [2] motivate construction-time changes (e.g., sorted α -reachable graphs); our work instead targets the *search-time* hot path.

Query-path costs. A Vamana query is a best-first beam search: pop the closest frontier vertex, compute distances to its out-neighbors, mark them visited, and merge qualifying candidates into the frontier. Costs concentrate in (i) distance computation, (ii) visited-set membership, and (iii) frontier maintenance. Each is a known lever: visited sets are classically implemented as hash tables or epoch-stamped dense arrays; early termination heuristics trade recall for fewer distance computations; quantized traversal with full-precision reranking reduces bandwidth pressure when the working set exceeds cache.

3 System Overview

The implementation is a Cargo workspace of 15+ crates in four tiers: SIMD/vector primitives (`diskann-wide`, `diskann-vector`), core math and utilities (`diskann-linalg`, `diskann-utils`, `diskann-quantization`), the algorithm and storage layer (`diskann`, `diskann-providers`, `diskann-disk`, `diskann-label-filter`), and benchmarking/tooling (`diskann-benchmark*`, `diskann-tools`). The benchmark runner is driven by a declarative `job.json` and supports a `recall_target` summary: the best QPS whose recall meets the target, reported per thread count — the industry-standard QPS-at-recall operating point used throughout this paper. Data preparation for SIFT1M is scripted (`tools/prepare_sift1m.py`); all measurements in Section 5 use recall@10 over 10,000 queries with 3 repetitions per operating point, and we report the throughput recorded in the frozen raw rows (per-repetition values are preserved in the logs).

4 Query-Path Optimizations

The optimizations below were proposed, implemented, and measured by the RSI loop between 2026-09-14 and 2026-09-15. Unless noted, percentages are measured under the RSI harness on SIFT1M

(1M $\times$ 128d) with QPS-at-recall ≥ 0.95 as the primary metric.

O1: Epoch-stamped visited set, 16-bit stamps. We replace the hash-set visited structure with a dense array stamped per query; clearing is an $O(1)$ epoch bump, with an overflow set for sentinel ids. The first version (32-bit stamps) gave +10–15% single-thread QPS. Narrowing stamps to 16 bits halves the footprint (4MB $\rightarrow$ 2MB at 1M points) for cache residency; instrumented breakdowns show visited-set time -45% and beam-expansion time -19% . Cumulative effect of the query-path package: 1T QPS@0.95 2994.6 $\rightarrow$ 4641.1 (+55%), with latency down across the ladder (L= 35: $\sim 330 \mu\text{s} \rightarrow 215 \mu\text{s}$; L= 100: $\sim 480 \mu\text{s} \rightarrow 318 \mu\text{s}$) at identical recall and distance-computation counts; two-thread scaling improves to $1.83\times$ (8486.6 QPS@0.95).

O2: Adaptive early termination. An optional stop rule halts expansion when the frontier exceeds the current k -th distance by a configurable ratio after a minimum hop count. On SIFT1M at ratio 1.2 (committed default) it cuts $\sim 10\%$ of distance computations at L= 60 and $\sim 50\%$ at L= 200 with recall statistically unchanged. A finer sweep showed ratio 1.15 dominates near the recall knee (-8.7% to -25% comparisons) but caps the recall ceiling at 0.9817 (vs. 0.9947 at 1.2), so 1.2 is kept as the robust value.

O3: Exposed tuning levers. `prefetch_lookahead` (sweep {8,16,24}: default 8 is already optimal on the evaluation host) and `beam_width` (beam > 1 is a net loss at 1T: +8–25% comparisons, no recall gain) are exposed in `job.json` as documented levers rather than committed wins.

O4: Fat LTO. Enabling `lto=fat` for release builds adds cross-crate inlining on the call-heavy provider $\rightarrow$ distance-kernel path: +10–12% search QPS on a 10K A/B, identical comparisons/recall; release build time rises to ~ 6 minutes.

5 Evaluation

5.1 Setup

All cross-system numbers come from a single session (2026-09-16) on one cloud VM instance: 2 vCPU, 3.6 GB RAM + 8 GB swap, OpenCloudOS 9.4. Dataset SIFT1M (1M $\times$ 128, L2); recall@10; 10k queries; 3 repetitions; QPS is the throughput recorded in the frozen raw rows. Single-thread (1T) is the reference protocol; NSG’s search binary parallelizes queries over OpenMP with 2 threads and is labelled 2T. Raw records ship in `bench/results/*.jsonl`.

5.2 Main result: QPS at recall ≥ 0.95

Table 1 ranks systems at the recall ≥ 0.95 operating point.

5.3 Recall–QPS ladders

Figure 1 shows the recall–QPS curves reconstructed from the frozen raw measurements (each point is the median over repeats), and Table 2 gives the full operating ladders (1T; NSG 2T). DiskANN-RSI trails stock Vamana by 31% at Ls=16 and 19% at Ls=32, reaches -11% at Ls=64, and is within noise at Ls ≥ 128 ; recalls match to ± 0.001 at every rung, indicating identical graph quality — the delta is pure search-kernel speed. hnsplib leads the ≥ 0.95 band on this host; NSG trails throughout despite a 2-thread advantage.

System	QPS	recall@10	Op. point	Build	Peak mem
hnswlib-master-default	6077	0.9637	ef=64	238.6s	1.31 GB
hnswlib-stock (0.8.0)	6017	0.9636	ef=64	242.0s	1.31 GB
vamana (stock, $\alpha=1.0$)	5708	0.9667	Ls=64	255.0s	1.27 GB
hnswlib-variant4	5632	0.9617	ef=64	260.4s	1.31 GB
diskann-rsi ($\alpha=1.0$)	5078	0.9669	Ls=64	131.9s	1.28 GB
nsg (2T)	2643	0.9525	Ls=32	451s [†]	1.40 GB

Table 1: QPS at recall ≥ 0.95 , single-thread unless noted. [†]NSG additionally requires a kNN graph pre-computation of several hours (efanna, K=200).

System	rung: recall/QPS (low $\rightarrow$ high)			
vamana $\alpha=1.0$	Ls16 .826/15382	Ls32 .914/9705	Ls64 .967/5708	Ls128 .990/3178
	Ls256 .998/1756			
vamana $\alpha=1.2$	Ls16 .881/12513	Ls32 .949/7853	Ls64 .984/4534	Ls128 .996/2658
	Ls256 .999/1448			
diskann-rsi $\alpha=1.0$	Ls16 .824/10587	Ls32 .914/7897	Ls64 .967/5078	Ls128 .990/3090
	Ls256 .998/1719			
diskann-rsi $\alpha=1.2$	Ls16 .881/9398	Ls32 .949/6650	Ls64 .983/4164	Ls128 .996/2502
	Ls256 .999/1435			
hnswlib-stock	ef16 .801/16738	ef32 .903/10281	ef64 .964/6017	ef128 .989/3279
	ef256 .997/1764			
hnswlib-master-default	ef16 .801/16368	ef32 .904/10448	ef64 .964/6077	ef128 .989/3258
	ef256 .997/1792			
hnswlib-variant4	ef16 .817/15621	ef32 .907/9853	ef64 .962/5632	ef128 .986/3174
	ef256 .996/1728			
nsg (2T)	Ls16 .877/2929	Ls32 .952/2632	Ls64 .986/2099	Ls128 .996/1547
	Ls256 .999/1030			

Table 2: Recall@10/QPS ladders on SIFT1M (1T; NSG 2T; QPS as recorded in the frozen raw rows, 3 repetitions per operating point).

5.4 Thread scaling and build time

At Ls=64, moving to two threads scales stock Vamana $1.75\times$ ($5708 \rightarrow 9993$ QPS) and DiskANN-RSI $1.87\times$ ($5078 \rightarrow 9472$); at Ls=16 the ratios are $1.81\times$ and $1.84\times$. DiskANN-RSI builds its index $1.9\times$ faster than stock Vamana (131.9s vs. 255.0s) and $1.8\times$ faster than hnswlib, at equal graph quality (matched recalls).

5.5 Divergence from the optimizer’s harness

The RSI harness reported +55%; the unified benchmark shows -9% to -31% at low search depths and parity at high recall. Three factors explain the gap. (i) *Different protocols*: the RSI harness used its own data layout, baseline pairing, and early-stop configuration; its baseline (pre-optimization source) is not bit-comparable to the stock C++ Vamana baseline used here. (ii) *Hardware regime*: on this 2-vCPU host the 1M-vector set (512MB) and the graph stream from DRAM, while the 2–4 MB visited set is cache-resident under either stamp width, so halving the 16-bit stamps has little room to act, while the diversity-phase code added to the kernel imposes constant per-expansion overhead that dominates at small L. (iii) *LTO asymmetry*: the RSI binary carries fat LTO (+10–12%), i.e., the reported deficit is measured *despite* an exclusive build-level advantage. The honest

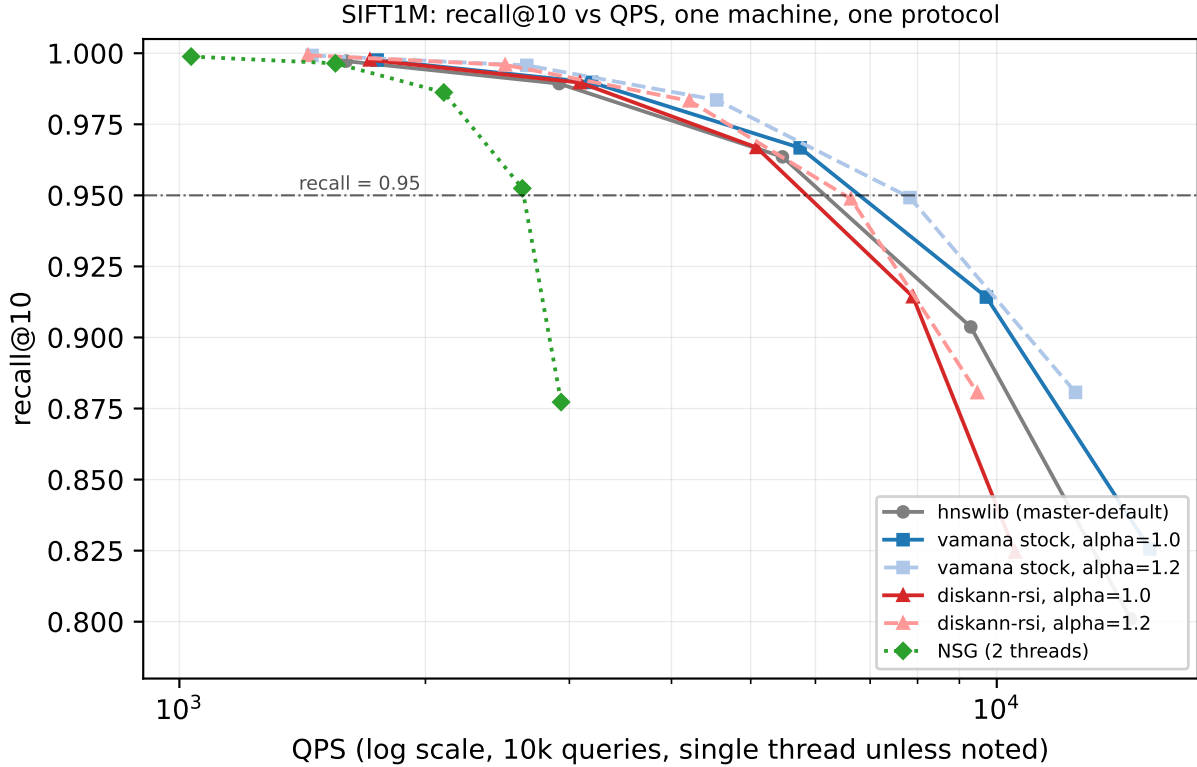


Figure 1: Recall@10 versus QPS on SIFT1M, measured in one session on one machine: hnswlib (master-default), stock Vamana, DiskANN-RSI and NSG (NSG at two threads, all others at one). Each marker is the median over repeats at one operating point of the raw rows in `bench/results/`; curves are reconstructed by `bench/plots/recall_qps.py`. The $\alpha=1.0$ DiskANN-RSI curve sits below stock Vamana at low search depths and converges to it at high recall, while $\alpha=1.2$ closes most of that gap.

summary is that O1–O4 are real but regime-dependent; the durable, regime-independent wins are build time (1.9 $\times$) and 2T scaling.

6 Negative Results

The RSI loop also produced measured rejections, which we record to save others the compute: **SQ8+rerank** tracks full precision within ~ 0.001 recall but yields no QPS gain here (5238 vs. 4641 at the metric point) — quantization pays when vector-fetch bandwidth is the bottleneck, which it is not on this compute-bound host; **candidate-queue merge insertion** was slower and reverted; **random-projection data reordering** had no measurable effect at 1M on this host; **partial-distance early exit** aborts 86.9% of distance computations but after ~ 2.9 of 4 chunks, a net loss for 128-dim vectors with a 4-chunk kernel; **beam width** > 1 inflates comparisons 8–25% with no recall gain at 1T; **early-stop ratio 1.15** caps the recall ceiling at 0.9817. Separately, an hnswlib pruning-heuristic variant (variant4) did not beat its same-source default at the ≥ 0.95 band (5632 vs. 6077 QPS).

7 Related Work

Beyond the graph-ANN literature of Section 2, this work sits in automated research tooling: The AI Scientist [7] and v2 [8] automate idea generation, experimentation, and writeup; Agent Laboratory [9] structures literature review, code solving, and report writing as collaborating agents. Our RSI loop is of the same family; this paper documents, with controlled re-measurement, the failure mode of accepting optimizer-harness metrics at face value — complementing benchmarking methodology work such as ANN-Benchmarks [6].

8 Conclusion

A Rust-native Vamana index with an automatically optimized query path wins decisively inside its optimizer’s harness (+55%) and roughly ties stock Vamana under unified same-machine conditions, with regime-independent wins in build time ($1.9\times$) and two-thread scaling ($1.87\times$). The gap between the two evaluations is itself a result: automated optimizers need protocol-unified, cross-system validation gates before their improvements are claimed as such. We release the code, the raw measurements, and the runbook to make that gate cheap to apply.

Reproducibility

The artifact is available at <https://github.com/tersemind/diskann-rsi> under the MIT License. Code, `job.json` configurations, raw JSONL measurements (`bench/results/`), and the environment runbook are included in the repository. The full optimization diff relative to the upstream port is in `docs/final_patch.diff`.

References

- [1] S. J. Subramanya, F. Devvrit, R. Kadekodi, R. Krishnaswamy, H. V. Simhadri. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. *NeurIPS*, Dec. 2019.
- [2] S. Gollapudi, R. Krishnaswamy, K. Shiragur, H. Wardhan. Sort Before You Prune: Improved Worst-Case Guarantees of the DiskANN Family of Graphs. *ICML*, PMLR 267:19787–19808, Jul. 2025.
- [3] Y. A. Malkov, D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE TPAMI* 42(4):824–836, Apr. 2020. arXiv:1603.09320, Mar. 2016.
- [4] C. Fu, C. Xiang, C. Wang, D. Cai. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *PVLDB* 12(5):461–474, Jan. 2019. arXiv:1707.00143, Jul. 2017.
- [5] P. Indyk, H. Xu. Worst-case Performance of Popular Approximate Nearest Neighbor Search Implementations: Guarantees and Limitations. *NeurIPS Datasets and Benchmarks*, 2023. arXiv:2310.19126.
- [6] M. Aumüller, E. Bernhardsson, A. Faithfull. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87, 2020.

- [7] C. Lu, R. T. Q. Chen, I. Lillich, et al. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292, Aug. 2024.
- [8] Y. Yamada, R. T. Q. Chen, et al. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv:2504.08066, Apr. 2025.
- [9] S. Schmidgall, Y. Su, Z. Wang, et al. Agent Laboratory: Using LLM Agents as Research Assistants. arXiv:2501.04227, Jan. 2025.